

27-linux-terminal

27. Linux Terminal & Shell Scripting

Level: □ Beginner

Jam: 8 (4 minggu × 2 sesi)

Prasyarat: Dasar komputer, bisa navigasi file

Output: Kumpulan shell script otomatisasi + monitoring

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Navigasi & manipulasi file system Linux (cd, ls, cp, mv, rm, find, permissions)
- Manage proses & networking (ps, top, kill, pipes, redirects, curl, SSH, tmux)
- Text processing pake grep, sed, awk, cut, sort, uniq, jq
- Nulis bash script dari nol (variable, loop, if/else, function, error handling)
- Bikin cron job, alias, dan script otomatisasi sendiri

Materi

Sesi	Topik	File
1	File System & Permission	01-file-system.md
2	Process & Networking	02-process-networking.md
3	Text Processing	03-text-processing.md
4	Shell Scripting	04-shell-scripting.md

Output Akhir Modul

Shell Script Automation — backup script, log rotator, deploy script, monitoring dashboard terminal

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain this bash command line by line"
- "Find the bug in this shell script"
- "Refactor this one-liner to a readable script"
- "Generate 3 test cases for this bash function"
- "Convert this Python snippet to a bash script"
- "Why does this cron job not run?"

01. File System & Permission

Navigasi Dasar

```
# cek posisi sekarang
pwd

# list file & folder
ls
ls -l          # detail (permission, size, date)
ls -a          # include hidden files
ls -lh         # human readable size
ls -la /tmp    # kombinasi

# pindah direktori
cd /home
cd ..          # naik 1 level
cd ~           # ke home
cd -           # ke direktori sebelumnya
```

Manipulasi File & Folder

```
# buat folder
mkdir project
mkdir -p project/src/components # nested

# copy
cp file.txt backup.txt
cp -r folder/ backup-folder/    # recursive

# move / rename
mv file.txt newname.txt
mv file.txt backup/             # pindah
```

```
# hapus
rm file.txt
rm -rf folder/ # recursive force – hati-hati!
```

File Permission

```
# lihat permission
ls -l script.sh
# -rwxr-xr-- 1 user group 1234 Apr 10 12:00 script.sh
# ^^rwxr-x--
# | | | | | | | |
# | | | | | | | | +-- other: read
# | | | | | | | | +--- other: --
# | | | | | | | | +---- group: read, exec
# | | | | | | | | +----- group: --
# | | | | | | | | +----- owner: read, write, exec
# | | | | | | | | +----- sticky bit
# | | | | | | | | +----- setgid
# | | | | | | | | +----- setuid / file type

# ubah permission (chmod)
chmod 755 script.sh # rwxr-xr-x
chmod u+x script.sh # owner + execute
chmod g-w file.txt # group - write
chmod o+r file.txt # other + read

# ubah owner (chown – pake sudo)
sudo chown user:group file.txt
sudo chown -R user:group folder/
```

Mencari File

```
# find – cari file berdasarkan nama, size, waktu
find . -name "*.log"
find /var -type f -size +100M
find ~ -mtime -7 # dimodifikasi 7 hari terakhir
find . -empty # file/folder kosong

# du – ukuran file/folder
du -sh folder/
du -h --max-depth=1 .

# df – sisa disk
df -h
df -h /home
```

Latihan

Latihan 1: Navigasi & Struktur Folder

Buat struktur folder proyek berikut dalam satu command:

```
project/  
├── src/  
│   ├── components/  
│   └── utils/  
├── tests/  
└── docs/
```

```
mkdir -p project/{src/{components,utils},tests,docs}
```

Latihan 2: File Permission

1. Buat file `deploy.sh`
2. Isi dengan `#!/bin/bash\necho "Deploying..."`
3. Ubah permission jadi executable buat owner, readable & executable buat group, readable aja buat other
4. Jalankan `./deploy.sh`

```
touch deploy.sh  
echo '#!/bin/bash  
echo "Deploying..."' > deploy.sh  
chmod 755 deploy.sh  
./deploy.sh
```

Latihan 3: Cari File Besar

Cari 5 file terbesar di `/var/log/`, tampilkan ukuran & path-nya.

```
du -ah /var/log/ | sort -rh | head -5
```

Latihan 4: Backup Otomatis (Manual)

1. Buat folder `source/` dengan 3 file dummy
2. Buat folder `backup/`
3. Copy semua file dari `source/` ke `backup/` pake `cp` + wildcard
4. Ganti nama folder `backup/` jadi `backup-$(date +%Y%m%d)`
5. Cek hasilnya

```
mkdir source backup
touch source/file{a,b,c}.txt
cp source/* backup/
mv backup/ "backup-$(date +%Y%m%d)"
ls -la
```

02. Process & Networking

Melihat Proses

```
# proses saat ini
ps aux
ps aux | grep nginx

# snapshot real-time
top
htop          # prettier version (kalau ada)

# proses spesifik
pgrep nginx   # dapet PID
pidof nginx   # alternatif

# tree proses
pstree
```

Mengontrol Proses

```
# kill berdasarkan PID
kill 1234
kill -9 1234      # force kill (SIGKILL)
kill -15 1234     # graceful (SIGTERM)
kill -HUP 1234    # reload config

# kill berdasarkan nama
pkill nginx
killall node

# background & foreground
sleep 100 &      # jalan di background
jobs             # lihat background jobs
fg %1            # foreground kan job 1
bg %1            # background kan job 1

# nohup – lanjut jalan walau terminal ditutup
```

```
nohup python server.py > server.log 2>&1 &
```

Pipes & Redirects

```
# redirect stdout ke file
echo "hello" > file.txt          # overwrite
echo "world" >> file.txt         # append

# redirect stderr
command 2> error.log
command 2>&1                     # gabung stderr ke stdout

# pipe – output jadi input perintah lain
ls -la | grep ".md"
ps aux | sort -nrk 3 | head     # proses paling boros CPU
cat access.log | awk '{print $1}' | sort | uniq -c | sort -nr

# /dev/null – buang output
command > /dev/null 2>&1
```

Networking Dasar

```
# curl – transfer data dari URL
curl https://api.github.com/users/nousresearch
curl -o file.html https://example.com
curl -X POST -d '{"key":"value"}' -H "Content-Type: application/json" https://ap

# wget – download file
wget https://releases.ubuntu.com/24.04/ubuntu-24.04-desktop-amd64.iso
wget -c https://...           # lanjutkan download yang terputus

# cek koneksi
ping google.com
ping -c 4 8.8.8.8

# cek port terbuka
ss -tuln                     # modern (netstat pengganti)
netstat -tuln                # legacy

# DNS
nslookup google.com
dig google.com
```

SSH & Remote

```
# login ke server
ssh user@192.168.1.100
ssh -p 2222 user@host

# copy file via SSH
scp file.txt user@host:/home/user/
scp -r folder/ user@host:/home/user/

# SSH key
ssh-keygen -t ed25519 -C "email@example.com"
ssh-copy-id user@host

# config SSH biar gampang
# ~/.ssh/config
# Host myserver
#     HostName 192.168.1.100
#     User midory
#     Port 2222
#     IdentityFile ~/.ssh/mykey
```

Tmux — Terminal Multiplexer

```
# mulai session
tmux new -s myproject

# detach: Ctrl+b d
# list session
tmux ls

# attach ke session
tmux attach -t myproject

# split pane: Ctrl+b " (horizontal), Ctrl+b % (vertikal)
# navigate pane: Ctrl+b arrow
# new window: Ctrl+b c
# rename window: Ctrl+b ,
```

Latihan

Latihan 1: Monitor Proses

Buat satu baris command untuk menampilkan 5 proses dengan penggunaan memori terbanyak.

```
ps aux --sort=-%mem | head -6
```

Latihan 2: Pipe Chain

Dari file `/var/log/syslog` (atau `access.log`), ekstrak:

1. Ambil baris yang mengandung "error" (case-insensitive)
2. Ambil kolom pertama (timestamp/IP)
3. Urutkan
4. Hitung unik
5. Tampilkan 10 teratas

```
grep -i "error" /var/log/syslog | awk '{print $1}' | sort | uniq -c | sort -nr |
```

Latihan 3: Cek Kesehatan Server

Bikin script satu baris yang ngecek:

- Apakah port 80 (HTTP) terbuka?
- Berapa rata-rata response time?
- Bandingin sama port 443 (HTTPS)

Pake curl dan ping.

```
echo "HTTP:" && curl -o /dev/null -s -w "Time: %{time_total}s\n" http://example.
```

Latihan 4: SSH + Tmux Workflow

Buat sesi tmux bernama "dev" dengan 2 pane:

1. Pane kiri: SSH ke server remote, jalanin htop
2. Pane kanan: local, siap buat nulis script

Tulis langkah-langkahnya.

```
tmux new -s dev
# Ctrl+b % -> split vertikal
# pane kiri: ssh user@host lalu htop
# pane kanan: vim script.sh
```

03. Text Processing

grep — Cari Teks

```
# basic
grep "error" app.log
grep -i "warning" app.log      # case-insensitive
grep -r "TODO" src/           # recursive
grep -l "config" *.py         # cuma nama file
grep -n "function" index.js   # tampilkan nomor baris
grep -c "failed" log.txt      # hitung jumlah

# regex
grep "^2024-" log.txt          # baris diawali "2024-"
grep "error\|fatal\|panic" log.txt
grep -E "[0-9]{3}\.[0-9]{3}"  # extended regex (ERE)
grep -P "\d{3}\.\d{3}"        # perl regex (kalau support)

# inverse & context
grep -v "debug" log.txt        # kebalikan
grep -A 2 "error" log.txt      # 2 baris setelah
grep -B 2 "error" log.txt      # 2 baris sebelum
grep -C 3 "error" log.txt      # 3 baris konteks
```

sed — Stream Editor

```
# substitusi
sed 's/lama/baru/' file.txt    # ganti pertama tiap baris
sed 's/lama/baru/g' file.txt   # ganti semua (global)
sed -i 's/lama/baru/g' file.txt # in-place

# hapus baris
sed '/debug/d' file.txt        # hapus baris mengandung "debug"
sed '3,5d' file.txt            # hapus baris 3-5

# print baris tertentu
sed -n '10,20p' file.txt
sed -n '/error/p' file.txt     # = grep

# contoh umum
sed 's/ */ /g' file.txt        # compress spasi
sed 's/^[[:space:]]*///' file.txt # trim leading whitespace
sed '/^$/d' file.txt           # hapus baris kosong
```

awk — Pemrosesan Kolom

```
# basic – print kolom
awk '{print $1, $3}' file.txt
awk '{print NF, NR, $0}' file.txt      # jumlah kolom, nomor baris, seluruh baris

# delimiter kustom
awk -F: '{print $1}' /etc/passwd      # split by ":"
awk -F',' '{sum+=$3} END {print sum}' data.csv

# filter & hitung
awk '$3 > 50 {print $1, $3}' scores.txt
awk '/error/ {count++} END {print count}' log.txt

# format output
awk '{printf "%-20s %10d\n", $1, $3}' data.txt
```

cut, sort, uniq, wc

```
# cut – ambil kolom per karakter/delimiter
cut -d: -f1,3 /etc/passwd
cut -c1-10 file.txt                  # karakter 1-10

# sort – urutkan
sort file.txt
sort -n file.txt                     # numeric
sort -r file.txt                     # reverse
sort -k2 -n data.txt                 # berdasarkan kolom 2

# uniq – hilangkan duplikat (HARUS di-sort dulu)
sort file.txt | uniq
sort file.txt | uniq -c              # hitung kemunculan
sort file.txt | uniq -d              # cuma duplikat

# wc – word count
wc -l file.txt                       # jumlah baris
wc -w file.txt                       # jumlah kata
wc -c file.txt                       # jumlah bytes
```

head, tail & Pipe Chain

```
# head/tail
head -20 file.txt                    # 20 baris pertama
tail -50 file.txt                    # 50 baris terakhir
tail -f app.log                      # follow – nunggu output baru
```

```
# pipe chain – contoh konkret
# 1. Cari 5 IP paling sering request dari access log
cat access.log | awk '{print $1}' | sort | uniq -c | sort -nr | head -5

# 2. Cari endpoint API paling lambat
cat access.log | awk '{print $7, $NF}' | sort -k2 -rn | head -10

# 3. Honeypot – filter IP mencurigakan
cat auth.log | grep "Failed password" | awk '{print $(NF-3)}' | sort | uniq -c
```

jq — JSON Parser

```
# basic
echo '{"name":"Alice","age":30}' | jq '.'
echo '{"name":"Alice","age":30}' | jq '.name'

# array
curl -s https://api.github.com/users/nousresearch/repos | jq '[][.name]'
curl -s https://api.github.com/repos/nousresearch/hermes-agent | jq '{name, desc}'

# filter
cat data.json | jq 'select(.age > 25)'
cat data.json | jq '[][. | select(.status=="active")] | {name, email}'

# format
cat minified.json | jq '.' # prettify
jq -c '.' pretty.json    # minify
```

Latihan

Latihan 1: Log Analyzer — grep + awk + sort

Dari file `server.log` (buat dummy dulu):

```
2024-06-01 10:00:00 INFO Server started
2024-06-01 10:01:23 ERROR Connection timeout to db
2024-06-01 10:02:45 WARN Memory usage 85%
2024-06-01 10:03:12 ERROR Disk full on /dev/sda1
2024-06-01 10:04:00 INFO Request handled in 230ms
2024-06-01 10:05:30 ERROR Connection timeout to db
```

Tampilkan:

1. Semua ERROR beserta pesannya
2. Hitung berapa kali tiap ERROR muncul
3. Urutkan dari yang paling sering

```
grep "ERROR" server.log | awk -F'ERROR ' '{print $2}' | sort | uniq -c | sort -nr
```

Latihan 2: CSV Processing — awk + sort

File nilai.csv (buat sendiri):

```
nama,matematika,ipa,inggris
Alice,85,90,78
Bob,72,68,85
Charlie,90,95,92
Diana,60,75,70
```

Hitung rata-rata tiap siswa, urutkan dari tertinggi.

```
awk -F',' 'NR>1 {rata=($2+$3+$4)/3; print rata, $1}' nilai.csv | sort -nr
```

Latihan 3: JSON API — jq

Pakai curl ambil data dari public API (contoh: JSONPlaceholder), lalu:

1. Ambil semua judul post
2. Cari post yang userID = 1
3. Tampilkan post yang judulnya mengandung kata "eum"

```
curl -s https://jsonplaceholder.typicode.com/posts | jq '.[[] | select(.userId==1)]'
curl -s https://jsonplaceholder.typicode.com/posts | jq '.[[] | select(.title | toLower | contains("eum"))]'
```

Latihan 4: Pipe Chain — Data Pipeline

Dari access.log (buat dummy dengan format IP - - [date]
"METHOD /path HTTP/1.1" status bytes):

1. Ekstrak IP & path request
2. Hitung berapa request tiap IP
3. Tampilkan 3 IP paling banyak request
4. Simpan hasil ke file top-ips.txt

```
awk '{print $1, $7}' access.log | sort | uniq -c | sort -nr | head -3 > top-ips.txt
```

04. Shell Scripting

Dasar Bash Script

```
#!/bin/bash
# ^ shebang – interpreter

# variable
NAMA="Alice"
echo "Halo, $NAMA!"
echo "Halo, ${NAMA}!"           # sama, tapi jelas batasnya

# input
read -p "Masukkan nama: " nama
echo "Hai $nama"

# argument
echo "Script: $0"
echo "Argumen 1: $1"
echo "Argumen 2: $2"
echo "Jumlah argumen: $#"
```

Conditional & Loop

```
#!/bin/bash

# if/else
if [ -f "$1" ]; then
    echo "$1 adalah file"
elif [ -d "$1" ]; then
    echo "$1 adalah direktori"
else
    echo "$1 tidak dikenal"
fi

# operator umum
# [ -f file ] – file exists
# [ -d dir ] – dir exists
# [ -z str ] – string kosong
# [ "$a" = "$b" ] – string sama
# [ "$a" -eq "$b" ] – numeric sama
# [ "$a" -gt "$b" ] – numeric lebih besar

# for loop
```

```

for file in *.log; do
    echo "Memproses $file"
    gzip "$file"
done

# while loop – baca file per baris
while IFS= read -r line; do
    echo "Baris: $line"
done < file.txt

# until loop
count=5
until [ "$count" -eq 0 ]; do
    echo "$count"
    count=$((count - 1))
done

# case
case "$1" in
    start) echo "Starting...";;
    stop)  echo "Stopping...";;
    restart) echo "Restarting...";;
    *)      echo "Usage: $0 {start|stop|restart}";;
esac

```

Function

```

#!/bin/bash

# define
log() {
    local msg="$1" # local scope
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $msg"
}

error_exit() {
    echo "ERROR: $1" >&2
    exit 1
}

# panggil
log "Script dimulai"
log "Memproses file..."

# return value lewat stdout

```

```

get_date() {
    date '+%Y%m%d'
}
today=$(get_date)
echo "Today: $today"

# exit code
check_root() {
    if [ "$(id -u)" -ne 0 ]; then
        return 1
    fi
    return 0
}

if check_root; then
    echo "Root user"
else
    echo "Bukan root"
fi

```

Error Handling

```

#!/bin/bash

# stop script kalau ada error
set -e           # exit on error
set -u           # error kalau undefined variable
set -o pipefail  # error di pipe cascade

# trap - tangkap signal & cleanup
cleanup() {
    echo "Cleanup: menghapus temporary files..."
    rm -rf /tmp/myscript-*
}

trap cleanup EXIT
trap 'echo "Interrupted!"; exit 1' INT

# manual error check
if ! mkdir -p /var/backups; then
    echo "Gagal buat folder backup!" >&2
    exit 1
fi

```

Cron Job

```
# edit crontab
crontab -e

# format: menit jam hari tanggal bulan weekday perintah
# */5 * * * * /home/midory/scripts/backup.sh

# contoh
# tiap jam 2 pagi
0 2 * * * /home/midory/scripts/daily-backup.sh

# tiap menit ke-15 tiap 6 jam
15 */6 * * * /home/midory/scripts/health-check.sh

# tiap hari Senin jam 3 pagi
0 3 * * 1 /home/midory/scripts/weekly-report.sh

# log output cron
0 2 * * * /home/midory/scripts/backup.sh >> /var/log/backup.log 2>&1

# list cron
crontab -l

# nonaktifkan sementara (comment pake #)
```

bashrc & Alias

```
# edit ~/.bashrc atau ~/.bash_aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'
alias gs='git status'
alias gp='git push'
alias ..='cd ..'
alias ...='cd ../..'
alias cls='clear'

# fungsi di bashrc
mkcd() {
    mkdir -p "$1" && cd "$1"
}

# reload
source ~/.bashrc
```

```
# PATH
export PATH="$HOME/bin:$PATH"
export PATH="$HOME/.local/bin:$PATH"
```

Latihan

Latihan 1: Backup Script

Buat file backup.sh yang:

1. Menerima argumen: folder yang mau di-backup
2. Ngecek folder ada atau tidak
3. Bikin folder backup di ~/backups/ dengan nama folder-YYYYMMDD-HHMMSS
4. Copy semua isi folder ke folder backup
5. Log hasil ke ~/backups/backup.log
6. Exit dengan error code 1 kalau gagal

```
#!/bin/bash
set -euo pipefail

SOURCE="${1:-}"
if [ -z "$SOURCE" ] || [ ! -d "$SOURCE" ]; then
    echo "Usage: $0 <folder>" >&2
    exit 1
fi

BACKUP_DIR="$HOME/backups"
TIMESTAMP=$(date '+%Y%m%d-%H%M%S')
FOLDER_NAME=$(basename "$SOURCE")
DEST="$BACKUP_DIR/$FOLDER_NAME-$TIMESTAMP"

mkdir -p "$DEST"
cp -r "$SOURCE"/* "$DEST/"

echo "[$(date '+%Y-%m-%d %H:%M:%S')] Backup $SOURCE -> $DEST" >> "$BACKUP_DIR/backup.log"
echo "Backup selesai: $DEST"
```

Latihan 2: Log Rotator

Buat log-rotate.sh yang:

1. Cari semua file *.log di folder ~/logs/
2. Kalau ukuran file > 1MB, kompres pake gzip
3. File hasil kompres diberi nama file.log-YYYYMMDD-HHMMSS.gz

4. Hapus file .gz yang lebih tua dari 7 hari
5. Log semua aksi ke ~/logs/rotator.log

```
#!/bin/bash
set -euo pipefail

LOG_DIR="$HOME/logs"
ROTATE_LOG="$LOG_DIR/rotator.log"
DAYS=7

mkdir -p "$LOG_DIR"

for logfile in "$LOG_DIR"/*.log; do
    [ -f "$logfile" ] || continue
    size=$(stat -c%s "$logfile" 2>/dev/null || stat -f%z "$logfile" 2>/dev/null)
    if [ "$size" -gt 1048576 ]; then
        ts=$(date '+%Y%m%d-%H%M%S')
        mv "$logfile" "$logfile-$ts"
        gzip "$logfile-$ts"
        echo "[$(date)] Compressed: $logfile" >> "$ROTATE_LOG"
    fi
done

find "$LOG_DIR" -name "*.gz" -mtime +$DAYS -delete -print >> "$ROTATE_LOG"
echo "[$(date)] Rotator selesai" >> "$ROTATE_LOG"
```

Latihan 3: Deploy Script

Buat deploy.sh yang:

1. Clone/pull repo dari git@github.com:user/project.git ke ~/project
2. Kalau folder belum ada, clone. Kalau udah ada, pull.
3. Install dependencies (asumsi npm install atau pip install -r requirements.txt)
4. Restart service (asumsi systemctl restart myapp)
5. Rollback kalau gagal — simpan backup folder sebelum deploy
6. Kirim notifikasi (echo aja) sukses/gagal

```
#!/bin/bash
set -euo pipefail

REPO="git@github.com:user/project.git"
DIR="$HOME/project"
BACKUP_DIR="$HOME/deploy-backups"
```

```

TIMESTAMP=$(date '+%Y%m%d-%H%M%S')

rollback() {
    echo "[${TIMESTAMP}] DEPLOY FAILED! Rollback..." >&2
    if [ -d "$BACKUP_DIR/${TIMESTAMP}" ]; then
        rm -rf "$DIR"
        cp -r "$BACKUP_DIR/${TIMESTAMP}" "$DIR"
        echo "Rollback selesai"
    fi
    exit 1
}

trap rollback ERR

if [ -d "$DIR" ]; then
    cp -r "$DIR" "$BACKUP_DIR/${TIMESTAMP}"
    cd "$DIR" && git pull
else
    git clone "$REPO" "$DIR"
fi

cd "$DIR"
if [ -f "requirements.txt" ]; then
    pip install -r requirements.txt
elif [ -f "package.json" ]; then
    npm install
fi

# restart service
sudo systemctl restart myapp || true

echo "[${TIMESTAMP}] Deploy sukses ke $DIR"

```

Latihan 4: Monitoring Dashboard

Buat `monitor.sh` yang:

1. Tampilkan tabel berisi:
 - CPU usage (dari `/proc/stat` atau `top -bn1`)
 - RAM usage (dari `free -h`)
 - Disk usage (dari `df -h`)
 - 5 proses dengan CPU tertinggi
 - 5 proses dengan RAM tertinggi
2. Update setiap 5 detik (clear screen tiap loop)

3. Berhenti kalau ditekan Ctrl+C
4. Simpan snapshot ke file monitor-YYYYMMDD-HHMMSS.log kalau argumen --save diberikan

```
#!/bin/bash
set -euo pipefail

SAVE=false
if [ "${1:-}" = "--save" ]; then
    SAVE=true
    SNAPSHOT="$HOME/monitor-$(date '+%Y%m%d-%H%M%S').log"
fi

cleanup() {
    echo "Monitor dihentikan."
    exit 0
}

trap cleanup INT

while true; do
    clear
    echo "===== SYSTEM MONITOR ====="
    echo "Waktu: $(date)"
    echo ""
    echo "--- CPU Usage ---"
    top -bn1 | grep "Cpu(s)" | awk '{print $2"% user, " $4"% system, " $8"% idle}'
    echo ""
    echo "--- RAM Usage ---"
    free -h | awk '/Mem:/ {print "Total: " $2 " | Used: " $3 " | Free: " $4 " | Av'"
    echo ""
    echo "--- Disk Usage ---"
    df -h / /home 2>/dev/null | awk 'NR>1 {print $1 ": " $3 " / " $2 " (" $5 " use'"
    echo ""
    echo "--- Top 5 CPU Processes ---"
    ps aux --sort=-%cpu | head -6 | awk '{printf "%-5s %-5s %-5s %s\n", $2, $3"%'"
    echo ""
    echo "--- Top 5 RAM Processes ---"
    ps aux --sort=-%mem | head -6 | awk '{printf "%-5s %-5s %-5s %s\n", $2, $4"%'"
    echo ""
    echo "Tekan Ctrl+C untuk keluar"

    if $SAVE; then
    {
        echo "=== Snapshot $(date) ==="
        echo "CPU: $(top -bn1 | grep 'Cpu(s)' | awk '{print $2"%}')"
    }
    fi
done
```

```
        echo "RAM: $(free -h | awk '/Mem:/ {print $3 "/" $2}')"
    } >> "$SNAPSHOT"
fi

sleep 5
done
```